PROJECT REPORT

FOR

Foundations of Artificial Intelligence

Multi-Agent Campus Transport Management System

A PROJECT REPORT

Submitted by

2116240701316 - MOHAMED AARIS P

2116240701356 - NEIL DANIEL A



October, 2025

BONAFIDE CERTIFICATE

Certified that this project report " **Multi-Agent Campus Transport Management System** " is the bonafide work of **Mohamed Aaris P (2116240701316)** and **Neil Daniel A (2116240701356)** who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	
1	INTRODUCTION	
2	LITERATURE REVIEW	
3	PROPOSED SYSTEM	
	3.1 ARCHITECTURE DIAGRAM	
4	MODULES DESCRIPTION	
	4.1 MODULE 1: DRIVER APPLICATION	
	4.2 MODULE 2: STUDENT APPLICATION	
5	IMPLEMENTATION AND RESULTS	
	5.1 EXPERIMENTAL SETUP	
	5.2 RESULTS	
6	CONCLUSION AND FUTURE WORK	
7	REFERENCES	

ABSTRACT

MACTMS (Multi-Agent Campus Transport Management System) is an end-to-end college bus tracking and transit intelligence platform designed for Android devices. The system provides real-time GPS tracking of campus buses on a three-dimensional satellite map powered by CesiumJS, automated geofence-based arrival and departure detection, passenger headcount logging, AI-powered stop-to-bus assignment, and push notifications. All components are coordinated through a shared Supabase backend that hosts a PostgreSQL database with PostGIS spatial extensions.

The architecture is organised as five autonomous event-driven agents that communicate solely through database state changes, ensuring complete decoupling. The Telemetry Agent streams driver GPS coordinates; the Geofence Detection Agent uses PostGIS ST_DWithin queries to detect stop arrivals and departures; the Occupancy Tracking Agent maintains live passenger counts; the Notification Dispatch Agent delivers Firebase Cloud Messaging push alerts; and the Visualisation & Intelligence Agent renders the live map and AI recommendations for students.

Artificial intelligence modules implemented in pure Dart include Constraint Satisfaction Problem (CSP) solving with Minimum Remaining Values and least-loaded-first heuristics for optimal stop-to-bus assignment, A* pathfinding on a road graph for minimum-travel-time routing, and recency-weighted demand estimation for forecasting passenger boardings. Experimental evaluation on three Bangalore transit routes demonstrates reliable real-time tracking, balanced fleet utilisation, and responsive user interfaces. The system addresses key limitations of

conventional campus transport solutions by combining spatial database intelligence, multi-agent reactivity, and transparent AI decision-making.

Keywords: Multi-agent systems, Campus transport, Real-time GPS tracking, Constraint Satisfaction Problem, A* search, PostGIS geofencing, Flutter, Supabase.

INTRODUCTION

Campus transportation is a critical operational service in educational institutions that host large student and staff populations. Traditional college bus systems typically rely on fixed schedules, paper-based or static route information, and limited real-time visibility for passengers. Drivers lack decision-support tools for stop assignment under capacity constraints, while students receive little or no information about bus location, occupancy, or expected arrival times. These shortcomings lead to overcrowding on popular routes, under-utilisation of other vehicles, missed connections, and overall inefficiency.

Recent advances in mobile computing, cloud databases, spatial query engines, and artificial intelligence make it feasible to construct intelligent transit platforms that close these gaps. MACTMS (Multi-Agent Campus Transport Management System) is designed precisely for this purpose. It is a complete Android-based solution comprising two companion applications—a Driver App and a Student App—backed by a Supabase PostgreSQL database with PostGIS extensions. The system delivers live three-dimensional satellite visualisation of bus positions, automated geofence detection of stop arrivals and departures, continuous occupancy tracking, AI-driven stop-to-bus assignment, and push notifications.

A defining characteristic of MACTMS is its multi-agent architecture. Five autonomous, event-driven agents operate independently and communicate exclusively through changes in shared database state. No agent invokes another directly. This design yields a reactive, loosely-coupled system that is easy to extend and maintain. On the intelligence side, pure-Dart implementations of Constraint Satisfaction Problem (CSP) solvers and A* pathfinding provide transparent, auditable decision support for both drivers and students.

The remainder of this report is organised as follows. Chapter 2 surveys related work in intelligent transportation systems, multi-agent architectures, and AI techniques for vehicle routing and assignment. Chapter 3 presents the proposed system architecture and data-flow model. Chapter 4 describes the two principal software modules in detail. Chapter 5 discusses the experimental setup and observed results. Chapter 6 concludes the work and outlines future enhancements. Chapter 7 lists the cited references.

LITERATURE REVIEW

Intelligent Transportation Systems (ITS) have been an active research area for several decades. Early systems focused on fixed-route schedule adherence and simple Automatic Vehicle Location (AVL) using GPS. More recent platforms incorporate real-time passenger information, demand-responsive routing, and predictive analytics. Mobile applications such as Citymapper, Moovit, and various university-specific bus trackers demonstrate the consumer demand for live transit data. However, most campus-oriented solutions remain limited to map display of vehicle positions and do not integrate capacity-aware assignment or multi-agent coordination.

Multi-agent systems (MAS) provide a natural paradigm for modelling distributed transportation entities. Agents representing vehicles, stops, passengers, and control centres can react to local events while contributing to global objectives. Classic work on agent-based traffic simulation and fleet management shows that event-driven, database-mediated coordination can achieve robustness without centralised orchestration. MACTMS adopts this philosophy by treating GPS telemetry, geofence evaluation, occupancy accounting, notification delivery, and visualisation as independent agents that interact solely through PostgreSQL row changes and Supabase Realtime channels.

Constraint Satisfaction Problems have long been applied to resource allocation and scheduling. Variable-ordering heuristics such as Minimum Remaining Values (MRV) and value-ordering strategies such as least-constraining or load-balancing values are standard techniques for improving search efficiency. In the transit domain, CSP formulations have been used for crew rostering and vehicle assignment. MACTMS formulates stop-to-bus assignment as a CSP whose

variables are stops, domains are available buses, and constraints encode capacity, sequence order, and coverage requirements.

Pathfinding algorithms, particularly A* search with admissible heuristics, are foundational for vehicle routing. When the underlying graph encodes road segments with travel-time costs, A* yields optimal minimum-time paths. Combining A* with live GPS deviation detection enables continuous re-planning. Demand estimation techniques that weight historical observations by recency improve the quality of capacity constraints fed into the CSP solver. Spatial database extensions such as PostGIS supply efficient geodetic distance predicates (ST_DWithin) that support reliable geofencing without continuous application-side polling.

The literature therefore supplies mature building blocks—multi-agent coordination patterns, CSP and A* algorithms, spatial SQL, and mobile real-time frameworks—yet few published campus systems integrate all of them into a single, fully operational Android platform. MACTMS fills this gap by delivering a production-ready multi-agent architecture that unifies real-time tracking, AI decision support, and passenger-facing visualisation.

PROPOSED SYSTEM

MACTMS is conceived as an end-to-end college bus tracking and transit intelligence platform. Its primary goals are: (i) continuous, low-latency visibility of bus positions on a three-dimensional satellite map; (ii) automatic detection of arrivals at and departures from geofenced stops; (iii) accurate, real-time passenger occupancy accounting; (iv) optimal assignment of stops to buses under capacity and sequencing constraints; (v) proactive push notifications to subscribed students; and (vi) transparent visualisation of the AI reasoning process for educational and debugging purposes.

The system is realised as two Flutter Android applications sharing a common Supabase backend. The Driver App is used by bus operators to select operating slots, invoke the CSP planner, start and end trips, broadcast GPS telemetry, and enter boarding counts. The Student App provides live map tracking, occupancy indicators, AI stop cards, load-balancing recommendations, route subscriptions, and proximity alerts. Both applications embed a CesiumJS WebView for three-dimensional visualisation.

All persistent state resides in a PostgreSQL database augmented with the PostGIS extension. Row-level security policies restrict write access according to user roles while permitting broad read access for live tracking. Database triggers implement the Geofence Detection, Occupancy Tracking, and Notification Dispatch agents, ensuring that critical logic executes inside the data tier and remains consistent regardless of client behaviour.

3.1 Architecture Diagram

The overall system architecture comprises three logical layers: the Supabase backend, the Driver App, and the Student App. Communication occurs over REST/RPC for transactional operations and WebSocket channels for real-time replication.

Inside the backend, PostgreSQL stores all core and AI-extension tables. PL/pgSQL trigger functions react to INSERT and UPDATE events on bus_positions, stop_boardings, and trips. A Deno Edge Function named send-push exchanges service-account credentials for an OAuth2 token and invokes the Firebase Cloud Messaging HTTP v1 API. Supabase Realtime publishes row changes to subscribed clients. The five runtime autonomous agents and their interaction pattern are summarised below:

Agent	Implementation	Trigger	Action
Telemetry Agent	Driver App GPS service	Driver starts trip	Streams GPS to bus_positions every 10 m
Geofence Detection Agent	geofence_trigger.sql	bus_positions INSERT/UPDATE	ST_DWithin detection; arrived/departed events
Occupancy Tracking Agent	occupancy_trigger.sql	stop_boardings INSERT	Increments trips.current_occupancy
Notification Dispatch Agent	notification_setup.sql + Edge Function	trips.running_status UPDATE	Queries FCM tokens; dispatches push
Visualisation & Intelligence Agent	Student App	Realtime WebSocket events	Renders 3D map, CSP card, proximity alerts

Event flow proceeds as follows. Driver GPS upserts reach bus_positions and fire the geofence trigger, which writes stop_events and may update running_status. An arrived event surfaces a headcount modal in the Driver App; the resulting boarding record increments occupancy via another trigger. Changes to running_status cause the notification trigger to call the Edge Function, which delivers FCM messages. Concurrently, Supabase Realtime streams the same row changes to the Student App, which updates the Cesium map and AI cards without polling.

The complete trip lifecycle begins with an administrator creating operating slots and associating available buses. A driver authenticates, selects a slot, invokes the CSP solver to produce stop assignments, starts a trip, and begins broadcasting GPS. Geofence events, headcount entry, occupancy updates, and notifications proceed automatically. A* recomputes the road path to the next assigned stop on trip start, stop advancement, or significant GPS deviation. The trip ends when the driver marks it completed.

MODULES DESCRIPTION

The software is partitioned into two primary modules corresponding to the two user populations: drivers and students. Shared pure-Dart AI libraries implement demand estimation, CSP assignment, road-graph construction, and A* search; identical copies reside in both application trees.

4.1 Module 1 – Driver Application

The Driver Application is a single-file Flutter Android program (approximately 60 KB of Dart source) that serves bus operators. After Supabase email/password authentication, the driver is linked to an existing bus via the driver_bus_assignments table. A slot picker presents the operating_slots available for the driver's route.

The AI Plan Stops feature invokes CspAssignmentService.solve(), which assigns each stop on the route to exactly one available bus while respecting capacity limits and sequence constraints. The resulting plan is written to the stop_assignments table. An AI panel subsequently displays the ordered list of assigned stops, the current next stop, and A* path statistics (distance in kilometres, estimated minutes, nodes explored).

When the driver taps Start Trip, a trips row is inserted with status in_progress, a foreground GPS service begins, and the first A* polyline is drawn on the Cesium globe. The GPS service upserts coordinates to bus_positions only when the vehicle has moved more than ten metres (Haversine check), thereby limiting database traffic. A* is recomputed on trip start, whenever the next assigned stop changes, or when the GPS position deviates more than 150 m from the expected path (throttled to once every fifteen seconds).

Arrival at a geofenced stop triggers a headcount numpad modal. The driver enters the number of boarding passengers (or zero/skip for unassigned stops). The value is inserted into stop_boardings, the occupancy trigger updates the trip total, and the next assigned stop becomes active. Ending the trip marks the record completed and stops the GPS service.

Key technical components include the geolocator package for continuous positioning, webview_flutter for the embedded CesiumJS map, flutter_local_notifications for the persistent foreground-service notification required by Android, and supabase_flutter for authentication, database access, and Realtime subscriptions.

4.2 Module 2 – Student Application

The Student Application targets students and staff. It supports both authenticated Supabase accounts and anonymous guest sessions. The primary interface is a full-screen CesiumJS three-dimensional satellite map that displays every active bus. Position updates arrive exclusively via Supabase Realtime WebSocket channels; no polling is performed.

Tapping a bus marker opens a details sheet containing the route timeline, live occupancy percentage, and an AI stop card. The stop card lists the stops assigned to the bus's current slot; departed stops are struck through and the next stop is highlighted. A load-balancing recommender suggests an alternative bus when the observed occupancy of the selected vehicle exceeds eighty percent of capacity.

Students may subscribe to one or more routes. Subscription records, together with device FCM tokens stored in user_devices, enable the Notification Dispatch Agent to deliver push messages when a trip's running status changes (on-time, late, or

arrived at College Gate). In-app proximity alerts are generated from Realtime stop_events when a bus approaches a stop relevant to the student.

An additional AI Simulation screen provides educational transparency. Three tabs allow step-by-step playback of the CSP solver (TRY, VALID, VIOLATION, BACKTRACK, SOLUTION events), node-by-node expansion of A* (with g, h, and f values), and an eleven-phase combined pipeline that sequences demand estimation, CSP assignment, per-leg A* routing, roadblock injection, and re-planning through to college arrival. The simulation consumes the identical pure-Dart libraries used in production, ensuring fidelity.

Supporting packages mirror those of the Driver App, with the addition of student-specific proximity logic based on the device's own location.

IMPLEMENTATION AND RESULTS

5.1 Experimental Setup

The backend was provisioned on a Supabase project with the PostGIS extension enabled. Core schema tables (users, routes, stops, buses, trips, bus_positions, stop_events, stop_boardings, etc.) were created first, followed by AI-extension tables (operating_slots, slot_buses, stop_assignments, road_nodes, road_edges). Row-level security policies were applied so that public read access is granted for tracking tables while write access is restricted to authenticated drivers who own the relevant trip.

Three Bangalore-based routes were seeded with realistic coordinates:

- Route A – Electronic City: 7 stops, 2 buses of capacity 40 each.
- Route B – Whitefield: 6 stops, 2 buses of capacity 35 each.
- Route C – Koramangala: 6 stops, 2 buses of capacity 30 each.

Fourteen buses across six operating slots were configured for the AI planning experiments. Historical stop_boardings data spanning fourteen days were synthesised to exercise the recency-weighted demand estimator (half-life of seven days, safety factor 1.1).

Both Flutter applications were built with Dart 3.11+ against the listed package versions (supabase_flutter ^2.17.1, geolocator ^14.0.3, webview_flutter ^4.10.0, etc.). CesiumJS was embedded via local HTML assets; an optional Cesium Ion access token enables higher-resolution imagery. The send-push Edge Function was deployed as a Deno TypeScript handler; a service-account placeholder remains for production FCM credentials.

SQL artefacts were executed in a strict order: schema, geofence trigger, occupancy trigger, notification setup, Realtime publication, RLS patches, seed data, and AI schema. Development devices included mid-range Android handsets running the latest stable Flutter channel.

5.2 Results

Real-time tracking performed reliably under the ten-metre movement threshold. GPS upserts appeared on the Student App Cesium map within the latency of the Supabase Realtime channel (typically sub-second on campus Wi-Fi). Geofence detection with a default one-hundred-metre radius correctly generated arrived and departed events; running_status transitions to on_time, late (arrival more than ten minutes after scheduled time), or arrived (College Gate) were observed as expected.

The CSP solver, employing MRV variable ordering and least-loaded-first value ordering, produced feasible assignments for all seeded slots. Up to two hundred solutions were enumerated and scored by the composite metric buses_used + max_utilisation + 0.5 × variance; the lowest-scoring plan was retained. Load balance across the fleet improved visibly compared with naïve sequential assignment. Trace recording enabled exact step-by-step replay in the AI Simulation screen, confirming correctness of the backtracking logic.

A* pathfinding on the production road graph (chain edges between consecutive stops plus skip-one edges) returned minimum-travel-time routes. Re-planning on GPS deviation greater than 150 m and on next-stop advancement occurred within the fifteen-second throttle. When an edge was marked is_blocked, the neighbour enumeration excluded it and A* automatically selected an alternative path. Visual

polylines obtained from the public OSRM service matched the decision graph sufficiently for driver guidance.

Occupancy accounting remained consistent: each boarding insert incremented current_occupancy by the exact count entered by the driver. Student-facing occupancy percentages and the eighty-percent load-balancing threshold functioned as designed. Push-notification delivery succeeded in laboratory tests once valid FCM credentials were supplied; the documented placeholder limitation was confirmed.

Known limitations observed during evaluation include: arrival prompts firing for every route stop rather than only CSP-assigned stops; road-graph fidelity limited to stop-chain topology rather than full street networks; and the necessity of manually synchronising the pure-Dart AI libraries between the two application trees. These points are recorded as future-work items.

CONCLUSION AND FUTURE WORK

MACTMS successfully demonstrates a practical multi-agent architecture for campus transit intelligence. By confining inter-agent communication to database state changes and Realtime channels, the system achieves strong decoupling while delivering a coherent end-to-end experience. Real-time GPS tracking on a three-dimensional map, automated geofencing, occupancy maintenance, AI-powered stop assignment, and push notifications operate together as a unified platform.

The pure-Dart implementations of demand estimation, CSP with MRV and load-balancing heuristics, and A* search provide transparent, educationally valuable decision support that can be inspected step-by-step. Evaluation on three realistic Bangalore routes confirmed functional correctness, acceptable latency, and improved fleet utilisation relative to non-intelligent baselines.

Several avenues for future enhancement are identified. Assignment-aware geofencing would suppress arrival prompts for stops skipped by the CSP plan. Replacing the stop-chain road graph with a higher-fidelity street network (or a self-hosted OSRM instance) would improve routing accuracy and remove dependence on the public demo server. Packaging the AI libraries as a shared Dart package would eliminate manual synchronisation. Scaling the CSP enumerator with time-bounded or heuristic-first pruning would accommodate larger slot sizes. Finally, completing the production FCM credential configuration and adding live blockage-reporting UI would close the remaining operational gaps.

Overall, MACTMS establishes a solid foundation for intelligent, multi-agent campus transportation systems and offers a clear roadmap for continued refinement.

REFERENCES

Russell, S. and Norvig, P. (2020) Artificial Intelligence: A Modern Approach, 4th ed., Pearson.

Hart, P. E., Nilsson, N. J. and Raphael, B. (1968) ‘A Formal Basis for the Heuristic Determination of Minimum Cost Paths’, IEEE Transactions on Systems Science and Cybernetics, Vol.4, No.2, pp.100-107.

Dechter, R. (2003) Constraint Processing, Morgan Kaufmann.

O’Sullivan, D. and Unwin, D. (2010) Geographic Information Analysis, 2nd ed., Wiley.

PostGIS Project Steering Committee (2024) PostGIS 3.4 Documentation. Available at: https://postgis.net/documentation/ (Accessed: 2025).

Supabase Inc. (2024) Supabase Documentation – Realtime and Edge Functions. Available at: https://supabase.com/docs (Accessed: 2025).

Google LLC (2024) Firebase Cloud Messaging HTTP v1 API. Available at: https://firebase.google.com/docs/cloud-messaging (Accessed: 2025).

Flutter Team (2024) Flutter Documentation. Available at: https://docs.flutter.dev (Accessed: 2025).

Cesium GS, Inc. (2024) CesiumJS Documentation. Available at: https://cesium.com/learn/cesiumjs/ (Accessed: 2025).

Wooldridge, M. (2009) An Introduction to MultiAgent Systems, 2nd ed., Wiley.

OSRM Project (2024) Open Source Routing Machine. Available at: http://project-osrm.org (Accessed: 2025).

Dijkstra, E. W. (1959) ‘A Note on Two Problems in Connexion with Graphs’, Numerische Mathematik, Vol.1, pp.269-271.